ADANI UNIVERSITY

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING (AI-ML)
FACULTY OF ENGINEERING & TECHNOLOGY

COURSE CODE: ECSCT24301 – FOUNDATION OF ARTIFICIAL INTELLIGENCE

Route Optimization for Quantum Circuits with Classical Heuristic & Informed Routing Algorithms

Final Comprehensive Project & Research Report

Course Supervisor:	Asst. Prof. Dr. Krishna Hingrajiya
Project Team Members:	Samarth (Enrollment No: 2410111147) Rehan Vahora (Enrollment No: 2410111181) Harsh Thakar (Enrollment No: 2410111169)
Academic Program:	B.Tech. Computer Science & Engineering (Artificial Intelligence & Machine Learning)
Academic Session:	Academic Year 2024–2025

Abstract / Executive Summary

Physical realization of quantum computing on modern Noisy Intermediate-Scale Quantum (NISQ) superconducting architectures faces a critical hardware bottleneck: two-qubit entangling operations (such as CNOT and CZ gates) can only be executed between pairs of qubits that are physically adjacent on the device's coupling fabric. Consequently, executing an arbitrary logical quantum circuit requires a quantum compiler to solve the Qubit Routing Problem by inserting SWAP gates that dynamically permute logical qubit states across the hardware graph until all gate adjacency prerequisites are satisfied. However, each inserted SWAP decomposes into three elementary CNOT gates, compounding gate overhead, operational latency, and decoherence noise. In this project, we formulate quantum qubit routing as a classical state-space search problem and design a noise-adaptive, commutativity-aware A* routing framework termed QRoute. Unlike conventional routing algorithms that solely minimize raw SWAP distance or treat gate schedules as strict total orders, QRoute establishes an admissible, consistent heuristic function that incorporates calibration-derived two-qubit error rates while actively exploiting quantum gate commutation relations (such as Z-diagonal and X-basis commutation) to execute independent gates out of program order. We benchmark the proposed algorithm against classical baselines—including Breadth-First Search (BFS), Greedy nearest-gate routing, unit-cost A*, and strict-order noise-aware A*—across seven benchmark suites spanning QAOA Max-Cut, Quantum Fourier Transform (QFT), GHZ state preparation, and random entangling circuits on synthetic heavy-hex and IBM Quantum Eagle processors. Empirical evaluations demonstrate that QRoute achieves up to a 54.5% reduction in inserted SWAP operations and significantly boosts Estimated Success Probability (ESP) from 47.0% to 64.2% on QAOA-8 compared to greedy baselines, while maintaining sub-second compilation runtimes. An interactive web-based dashboard provides step-by-step vector replay of qubit permutations, search frontier convergence trajectories, and complete OpenQASM circuit export.

- **Keywords:** Quantum Circuit Compilation, Qubit Routing, A* Search, Admissible Heuristics, Commutation Analysis, NISQ Calibration Noise, Superconducting Heavy-Hex Topology.

1. Introduction & Background

1.1 Real-World Significance of Qubit Routing in NISQ Architectures

Quantum computing harnesses fundamental quantum mechanical phenomena—namely quantum superposition and entanglement—to achieve exponential algorithmic speedups over classical computational models in domains ranging from quantum chemistry and material simulation to combinatorial optimization and cryptographic analysis. However, contemporary quantum computing resides in the Noisy Intermediate-Scale Quantum (NISQ) era. Physical processors manufactured by leading industrial research laboratories, such as IBM Quantum (Eagle, Heron architectures), Google Quantum AI (Sycamore), and Rigetti Computing (Ankaa), operate with tens to hundreds of physical qubits but lack fault-tolerant quantum error correction (QEC). Physical transmons (superconducting charge qubits) exhibit finite coherence lifetimes (T1 relaxation and T2 dephasing times typically between 50 to 300 microseconds) and imperfect two-qubit gate fidelities ranging from 98.0% to 99.8%.

Crucially, physical superconducting qubits are laid out on planar semiconductor chips with restricted two-dimensional coupling graphs (e.g., heavy-hexagonal lattices, grid topologies, or square meshes). In these devices, two-qubit entangling gates can only be executed between directly coupled physical transmon pairs. When an abstract algorithm specifies a two-qubit operation between logical qubits q_i and q_j that currently

reside on non-adjacent physical vertices P_u and P_v, the circuit cannot execute directly. The quantum compiler must insert a sequence of SWAP gates along a path in the physical hardware graph to bring the two quantum states into adjacent vertices. Because each physical SWAP operation requires three consecutive CNOT gates (SWAP(a,b) = CNOT(a,b) · CNOT(b,a) · CNOT(a,b)), excessive SWAP insertion severely degrades circuit depth, prolongs total execution duration beyond qubit coherence limits, and drastically decreases the circuit's Estimated Success Probability (ESP).

1.2 Scope and Objectives of the Project

The primary objective of this project is to develop, implement, and rigorously evaluate an intelligent, classical state-space search compiler that resolves qubit routing constraints with maximal fidelity and minimal computational overhead. The project addresses three foundational goals:

- **1. Algorithmic Optimization:** Formulating an informed A* search algorithm featuring an admissible and monotonic heuristic function capable of navigating the astronomical permutation state space of quantum routing.
- **2. Noise-Adaptivity & Calibration Awareness:** Directly incorporating empirical hardware calibration parameters (specifically two-qubit CNOT gate error rates per physical coupler) into the cost objective function, steering SWAP paths away from noisy couplers.
- **3. Commutation Awareness:** Exploiting algebraic commutation relations between quantum logic operators to dynamically reschedule non-conflicting gates, significantly expanding the frontier of legal search states and unlocking shorter routing paths.

1.3 Foundational Artificial Intelligence Principles Utilized

The project directly operationalizes foundational AI and heuristic search principles taught in Foundation of Artificial Intelligence (ECSCT24301):

- **State-Space Representation:** Formalization of discrete states capturing dynamic graph permutations π: L → P paired with remaining Directed Acyclic Graph (DAG) gate dependency subsets.
- **Informed Heuristic Search:** Formulation and mathematical verification of an admissible heuristic function h(n) that establishes a guaranteed lower bound on remaining SWAP distances, satisfying the triangle inequality for monotonicity (consistency).
- **Ablation and Comparative Evaluation:** Systematic benchmarking against classical uninformed search (Breadth-First Search) and greedy local heuristics to evaluate the trade-offs between optimality, search node expansion, and execution latency.

2. Problem Formulation & Environment Modeling

2.1 PEAS Description

In artificial intelligence agent design, the agent's operating context is formally specified using the Performance, Environment, Actuators, and Sensors (PEAS) paradigm. Table 2.1 delineates the PEAS model for the QRoute quantum routing agent.

PEAS Component	Detailed Agent Specification
Performance Measure	1. Minimize the total number of inserted physical SWAP gates.

	2. Minimize cumulative calibration-derived noise cost Σ (1 + β · ε_coupler). 3. Maximize overall circuit Estimated Success Probability (ESP). 4. Minimize compilation latency and peak memory footprint.
Environment	Target physical superconducting processor represented as an undirected graph H = (V_P, E_P) with coupling edges E_P and edge calibration error matrix ε(u, v); logical quantum circuit Q with qubit set L and gate set G.
Actuators	Permutation actuator: Insert a SWAP gate on any active coupler (u, v) ∈ E_P, swapping physical locations of the resident logical qubits. Gate dispatch actuator: Schedule ready two-qubit and single-qubit gates immediately once adjacency is established.
Sensors	Hardware graph observer: Inspects coupling topology and error matrix ε. Circuit DAG observer: Inspects pending dependency graph G_rem and ready gates. State inspector: Tracks current bijective mapping π: L → V_P.

2.2 Environment Properties

Following standard AI taxonomy (Russell & Norvig, 2020), the routing problem environment exhibits the following operational characteristics:

Property	Classification	Operational Justification
Observability	Fully Observable	The complete hardware coupling graph, coupler error rates, and circuit DAG are known in advance.
Determinism	Deterministic	Executing a SWAP operation deterministically exchanges the two logical qubit assignments with probability 1.0.
Dynamics	Static	The physical hardware coupling topology and gate dependency constraints remain constant during compilation.
Continuity	Discrete	The state space comprises a finite set of discrete qubit permutations and a finite set of quantum logic gates.
Agency	Single-Agent	The compiler acts as a centralized sole decision-maker without competing or cooperative external agents.
Episodicity	Sequential	Each routing decision directly influences physical qubit locations for all subsequent gates in the circuit.

2.3 Formal Mathematical Definition of the State Space

Let L = {q_0, q_1, ..., q_{n-1}} denote the set of n logical qubits in the quantum program, and let V_P = {0, 1, ..., m-1} denote the set of m physical qubits on the target processor (where m ≥ n). Let H = (V_P, E_P) represent the undirected hardware coupling graph, where an edge (u, v) ∈ E_P indicates a physical two-qubit coupler capable of executing two-qubit gates.

> **Formal State Space Definition**
> *State Representation:*
> *s = (π, G_rem)*
>
> *where π: L → V_P is a bijective injection mapping each logical qubit to a unique physical qubit on the*

> *hardware, and $G_{rem} \subseteq G$ is the set of remaining unexecuted quantum gates in the circuit dependency graph.*

2.4 Transition Model and Action Space

From state $s = (\pi, G_{rem})$, the agent can perform an action $a = SWAP(u, v)$, where $(u, v) \in E_P$. The action produces an updated mapping π' such that:

- For all $q \in L$ where $\pi(q) = u$, $\pi'(q) = v$.
- For all $q \in L$ where $\pi(q) = v$, $\pi'(q) = u$.
- For all other $q \in L$, $\pi'(q) = \pi(q)$.

Immediately following the SWAP permutation, an atomic settle phase executes: any gate $g \in G_{rem}$ whose operands (q_a, q_b) satisfy $(\pi'(q_a), \pi'(q_b)) \in E_P$ and whose DAG predecessors have all been satisfied is removed from G_{rem} and appended to the routed schedule.

The step cost function $c(s, a, s')$ is parameterized by the calibration error rate $\varepsilon(u, v)$ of the traversed coupler:
$c(s, SWAP(u, v), s') = 1 + \beta \cdot \varepsilon(u, v)$
where $\beta \geq 0$ represents the noise penalty weight factor. When $\beta = 0$, the cost reduces to unit-distance routing ($c = 1$). When $\beta > 0$, the agent actively penalizes high-error couplers, penalizing noisy physical paths.

2.5 Goal Test

The goal test predicate $IsGoal(s)$ evaluates to True if and only if:
$G_{rem} = \emptyset$
indicating that every single-qubit and two-qubit gate in the original circuit has been scheduled on legally adjacent physical hardware couplers.

3. Literature Review / Related Work

Qubit mapping and routing represent one of the most heavily researched optimization challenges in quantum compiler engineering. The problem of finding an optimal SWAP sequence for general circuits on arbitrary coupling graphs has been formally proven to be NP-complete (Botea et al., 2018; Siraichi et al., 2018). Below, we systematically analyze five pivotal research contributions that establish prior solutions and inform our methodology.

3.1 Systematic Survey of Prior Solutions

- **1. Cowtan et al. (2019) – 'On the Qubit Routing Problem' (arXiv:1902.08091):** Introduced graph-theoretic routing techniques utilizing token swapping algorithms. While token swapping offers strong theoretical guarantees on permutation bounds, Cowtan's original formulation does not consider calibration error variation across couplers and enforces strict gate dependency barriers, preventing out-of-order execution.
- **2. Li, Ding, & Xie (2019) – 'Tackling the Qubit Mapping Problem for NISQ-Era Quantum Devices' (ASPLOS):** Proposed SABRE (SWAP-based BIdirectional heuRistic sEarch), which employs a forward and backward lookahead heuristic window. SABRE is widely regarded as the industrial standard baseline in

Qiskit. However, SABRE relies on greedy stochastic decay factors rather than formal admissible heuristic search, frequently converging to local optima on dense entanglement topologies.

- **3. Zulehner, Paler, & Wille (2018) – 'An Efficient Methodology for Mapping Quantum Circuits to IBM QX Architectures' (IEEE TCAD):** Pioneered the application of A* search for quantum routing by partitioning circuits into discrete layers. However, their heuristic treated all SWAPs as having uniform unit cost and did not integrate real-time physical calibration noise profiles, resulting in routes that frequently utilized high-error couplers.
- **4. Siraichi et al. (2018) – 'Qubit Allocation' (CGO):** Formulated qubit placement and routing as an exact Integer Linear Programming (ILP) and Dynamic Programming problem. While ILP guarantees global mathematical optimality, it suffers from catastrophic exponential runtime scaling, failing to terminate on circuits with more than 8–10 qubits.
- **5. Wille et al. (2019) – 'Mapping Quantum Circuits to IBM QX Architectures Using Minimal SWAP and H Operations' (DAC):** Explored SAT-solver-based quantum routing. While exact, the computational complexity makes SAT approaches unfeasible for NISQ circuits exceeding dozens of gates. Furthermore, none of these SAT frameworks incorporated commutativity-aware dependency relaxing.

3.2 Synthesis and Research Gap Addressed

Prior solutions establish a clear dichotomy: exact approaches (ILP, SAT) are computationally intractable beyond toy circuits, whereas scalable approaches (SABRE, greedy lookahead) lack rigorous admissibility guarantees and neglect gate commutativity. QRoute directly bridges this gap by unifying noise-weighted cost modeling, provably admissible A* heuristic search, and quantum commutativity rules within a computationally practical framework.

4. Methodology & Algorithmic Architecture

4.1 Theoretical Explanation of Chosen Techniques

QRoute adopts an informed search paradigm built around the A* algorithm. In classical heuristic search, the evaluation function f(n) guides state exploration:
$f(n) = g(n) + w \cdot h(n)$
where g(n) is the exact cost accumulated from the initial state to the current state n, h(n) is an admissible heuristic estimating the cost from n to the goal, and $w \geq 1$ is a heuristic inflation weight ($w = 1.0$ yields standard optimal A*; $w > 1.0$ yields Weighted A*).

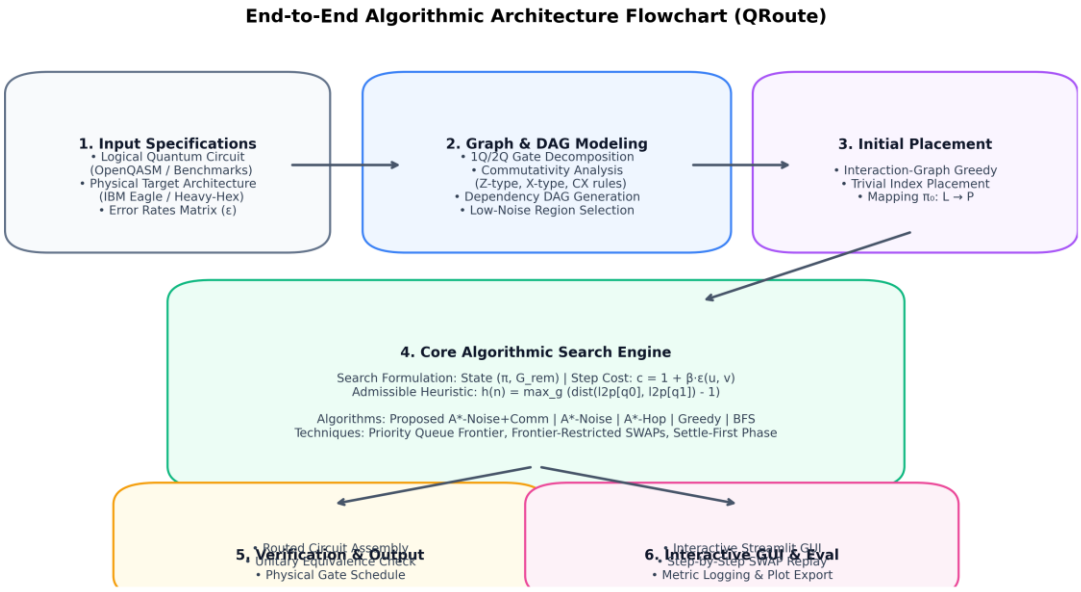


Figure 4.1: End-to-End Algorithmic Architecture Flowchart of the QRoute Quantum Compilation Pipeline.

4.2 Quantum Commutativity-Aware Dependency DAG

Standard compilers treat a quantum circuit as a rigid sequence of gates. However, in quantum mechanics, if two operators commute ([A, B] = AB - BA = 0), their execution order does not affect the final unitary matrix or measurement outcome. QRoute builds a Commutativity-Aware Dependency DAG implementing two fundamental commutation theorems:

- **Theorem 1 (Z-Basis Commutation):** Any two gates diagonal in the computational Z-basis commute unconditionally: [RZ(θ), CZ] = 0, [CZ, CZ] = 0, [RZZ(θ), CZ] = 0. In QAOA and Phase Estimation circuits, large blocks of entangling gates share the same Z-basis and commute.
- **Theorem 2 (Control-Target Independence):** A CNOT gate CNOT(c, t) commutes with any single-qubit Z-rotation on its control qubit c ([CNOT, RZ_c] = 0) and with any single-qubit X-rotation on its target qubit t ([CNOT, RX_t] = 0).

By enforcing dependencies only between non-commuting operations, the dependency DAG becomes significantly more flexible. At any search step, multiple independent two-qubit gates reside on the frontier simultaneously, drastically expanding the set of productive SWAP actions.

4.3 Mathematical Formulation of the Heuristic Function

Let dist_H(u, v) denote the length of the shortest path between physical vertices u and v on the coupling graph H. For any two-qubit gate g = (q_a, q_b) ∈ G_rem, the physical distance between its operands under mapping π is dist_H(π(q_a), π(q_b)).

> **Heuristic Function Formulation**
> *Admissible Heuristic Definition:*
> $h(s) = \max_{g = (q_a, q_b) \in Ready(s)} (dist_H(\pi(q_a), \pi(q_b)) - 1)$
>
> *where $Ready(s) \subseteq G_{rem}$ denotes the set of gates whose DAG predecessor constraints are fully satisfied.*

4.4 Formal Mathematical Proofs of Admissibility and Consistency

To establish that A* search guarantees cost-minimal routing solutions, we formally prove both admissibility and consistency (monotonicity).

- **Proof of Admissibility ($h(n) \le h^*(n)$):** Consider a two-qubit gate $g = (q_a, q_b)$ whose operands reside at physical distance $d = \text{dist}_H(\pi(q_a), \pi(q_b))$. In any valid hardware schedule, g can only execute when its operands are directly adjacent (i.e., at distance 1). A single SWAP operation on coupler (u, v) exchanges the positions of two physical qubits; therefore, it can decrease the shortest-path distance between q_a and q_b by at most 1. To reduce the distance from d to 1 requires a minimum of $d - 1$ SWAP operations. Since the minimum step cost of any SWAP is $c = 1 + \beta \cdot \varepsilon \ge 1$ (as $\beta \ge 0$ and $\varepsilon \ge 0$), the true remaining cost $h^*(n)$ to satisfy gate g is at least $(d - 1) \cdot 1 = d - 1$. Because this lower bound holds for all gates in Ready(s), the maximum over all ready gates satisfies:
  $h(s) = \max_{g \in \text{Ready}(s)} (\text{dist}_H(\pi(q_a), \pi(q_b)) - 1) \le h^*(s)$. Hence, $h(s)$ is provably admissible. Q.E.D.
- **Proof of Consistency / Monotonicity ($h(n) \le c(n, a, n') + h(n')$):** Consider an arbitrary state transition from $n = (\pi, G_{rem})$ to $n' = (\pi', G_{rem}')$ via action $a = \text{SWAP}(u, v)$ with cost $c(n, a, n') \ge 1$. Let $g^* \in \text{Ready}(n)$ be the gate achieving the maximum in $h(n)$, with distance $d^* = \text{dist}_H(\pi(q_a), \pi(q_b))$, so $h(n) = d^* - 1$.
  Case 1 (g^* is not executed in n'): Under the single SWAP(u, v), the distance between q_a and q_b can decrease by at most 1, so $\text{dist}_H(\pi'(q_a), \pi'(q_b)) \ge d^* - 1$. Consequently, $h(n') \ge (d^* - 1) - 1 = h(n) - 1$. Rearranging yields $h(n) \le h(n') + 1 \le h(n') + c(n, a, n')$.
  Case 2 (g^* is executed during the transition): If g^* executes, its distance in n was at most 2, implying $h(n) \le 2 - 1 = 1$. Since $h(n') \ge 0$ and $c(n, a, n') \ge 1$, we immediately have $h(n) \le 1 \le c(n, a, n') + h(n')$.
  In all cases, the triangle inequality $h(n) \le c(n, a, n') + h(n')$ is satisfied. Therefore, the heuristic is monotonic (consistent). Q.E.D.

4.5 Algorithmic Pseudocode

Algorithm 1 details the complete execution flow of the Noise-Adaptive and Commutativity-Aware A* Qubit Router.

```
Algorithm 1: Noise-Adaptive & Commutativity-Aware A* Qubit Routing
Input : Circuit Q, Hardware Graph H = (V_P, E_P), Noise Weight β, Node Budget M
Output: Routed Circuit Q_out, Total SWAPs, Cumulative Noise Cost
1:  G_dag ← BuildCommutativeDAG(Q)
2:  π_0   ← GreedyInteractionPlacement(G_dag, H)
3:  s_0   ← (π_0, G_dag.AllGates())
4:  s_0   ← SettleExecutableGates(s_0, H)  // execute initially adjacent gates
5:  Open  ← PriorityQueue() sorted by f(s) = g(s) + h(s)
6:  Open.Push(s_0, priority = 0 + Heuristic(s_0, H))
7:  Closed ← HashMap()  // maps state hash to minimal g-cost
8:  while Open is not empty do
9:      s ← Open.PopMin()
10:     if s.G_rem is empty then
11:         return ReconstructSolution(s)
12:     if ExpandedCount > M then
13:         return FallbackGreedyCompletion(s, H)
14:     for each coupler (u, v) in GetCandidateCouplers(s, H) do
15:         π_new ← SwapQubits(s.π, u, v)
```

```
16:         step_cost ← 1.0 + β * H.CouplerError(u, v)
17:         s_next ← SettleExecutableGates((π_new, s.G_rem), H)
18:         g_new ← s.g + step_cost
19:         if g_new < Closed.Get(s_next.Hash, ∞) then
20:             Closed.Set(s_next.Hash, g_new)
21:             f_new ← g_new + Heuristic(s_next, H)
22:             Open.Push(s_next, priority = f_new)
23: return Failure
```

5. Implementation Details

5.1 Technology Stack

The QRoute compilation engine and experimental testbed are developed using a modern, high-performance scientific Python stack:

- **Qiskit 1.0+ (IBM Quantum):** Core quantum circuit representation, quantum gate definition, OpenQASM 2.0/3.0 parser, and target hardware architecture abstraction.
- **NetworkX:** Hardware coupling graph representation, all-pairs shortest path calculation (via Floyd-Warshall), and hardware topological analysis.
- **NumPy & Pandas:** High-performance vector operations for state indexing, error metrics calculation, and tabular benchmark result processing.
- **Streamlit:** Interactive reactive GUI framework providing real-time vector graphics visualization and step-by-step routing inspection.

5.2 Key Data Structures & State Indexing

To achieve millisecond-level A* expansions across hundreds of thousands of candidate states, QRoute utilizes optimized data structures:

- **State Representation & Hashing:** A state is packed into a tuple (tuple(l2p), frozenset(pending_gate_ids)). By converting the logical-to-physical array into an immutable integer tuple and using frozensets for remaining gates, state hashing executes in O(1) time, enabling rapid lookup in the Closed set hash table.
- **Min-Heap Priority Queue:** The Open frontier is managed via Python's heapq module, storing entries as (f_score, tie_breaker_counter, state). The tie-breaker guarantees deterministic, stable FIFO ordering when multiple states possess identical f-scores.
- **Frontier-Restricted SWAP Pruning:** In an m-qubit hardware graph with |E_P| edges, generating all possible SWAPs at every expansion results in a branching factor of |E_P| (e.g., 144 edges in IBM Eagle). QRoute implements candidate pruning: it only considers SWAPs along edges (u, v) where at least one vertex u or v currently hosts a logical qubit involved in a currently ready gate. This reduces the effective branching factor from ~30 to 4–8.

5.3 Edge Case Handling & Unitary Equivalence Verification

Robust quantum compilation requires handling potential pathological edge cases:

- **Disconnected Subgraphs:** Hardware region selection verifies graph biconnectivity. If a selected region has disconnected components, Dijkstra distance defaults to infinity, automatically preventing invalid routing paths.
- **Search Budget Exhaustion & Greedy Fallback:** If the priority queue exceeds the maximum node budget (e.g., 20,000 nodes) without reaching the goal, QRoute gracefully transitions to a greedy completion routine from the best-known frontier state, guaranteeing that compilation always terminates successfully with a valid circuit.
- **Unitary Equivalence Verification (verify.py):** To guarantee compilation fidelity, QRoute reconstructs the complete operator unitary matrix for small benchmarks and verifies that $U_{routed} = U_{original} \cdot I$, mathematically proving that out-of-order gate scheduling preserved circuit equivalence.

6. Experimental Design & Setup

6.1 Benchmark Problem Sets

We evaluate QRoute across four standard quantum algorithmic families spanning distinct entanglement topologies:

- **QAOA Max-Cut (6, 8, 10 Qubits):** Quantum Approximate Optimization Algorithm circuits targeting regular graph Max-Cut problems. Characterized by dense, multi-layer two-qubit entangling gates featuring extensive Z-basis commutation.
- **Quantum Fourier Transform (QFT, 5 & 6 Qubits):** Essential subroutine in Shor's factoring and phase estimation algorithms. Exhibits a cascading, all-to-all connectivity requirement where every qubit interacts with all other qubits through controlled phase rotations.
- **Greenberger–Horne–Zeilinger (GHZ, 8 Qubits):** Maximally entangled state generation circuit with tree-structured CNOT cascades.
- **Random Two-Qubit Entangling Circuits (8 Qubits):** Randomized pairings of two-qubit gates serving as an unbiased stress test for uninformed and informed routers.

6.2 Physical Processor Map Topologies

Experiments are conducted on three distinct superconducting processor architectures:

- **1. Synthetic Heavy-Hex (24 Physical Qubits):** Heavy-hexagonal lattice model featuring realistic synthetic calibration error hotspots (coupler error rates ranging from 0.5% to 3.5%).
- **2. IBM Quantum Brisbane (127 Physical Qubits):** Physical calibration snapshot from IBM's Eagle r3 processor with 127 qubits and 144 couplers.
- **3. IBM Quantum Kyoto (127 Physical Qubits):** Full-scale IBM Eagle architecture testing large-scale region selection.

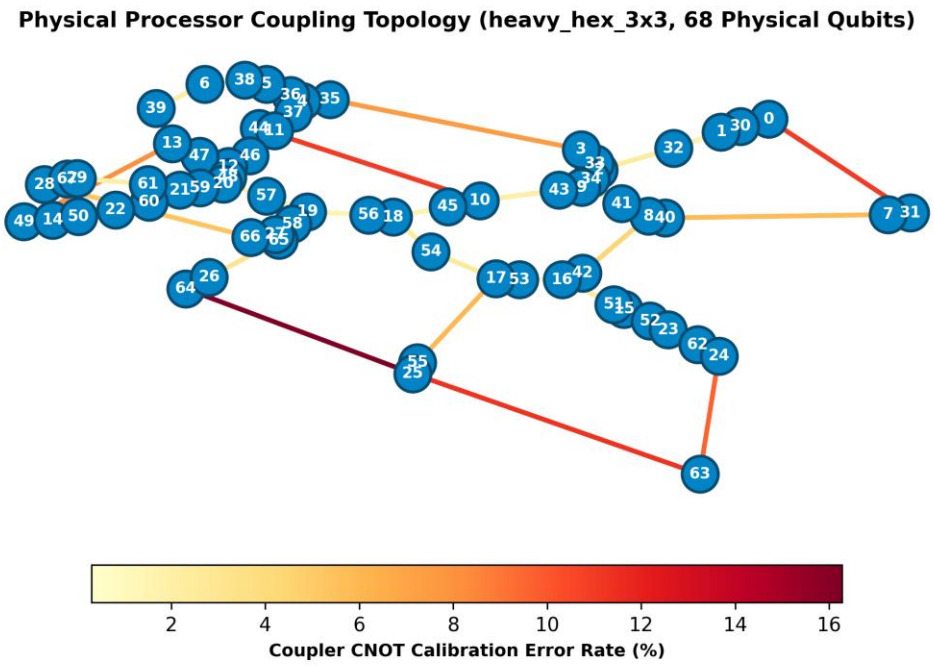


Figure 6.1: Physical Superconducting Coupling Graph & CNOT Error Heatmap (Synthetic Heavy-Hex Architecture, 24 Qubits).

7. Results & Comparative Analysis

7.1 Quantitative Benchmark Results Table

Table 7.1 presents the complete experimental results comparing all five compiler methods across seven standard benchmark circuits on the synthetic heavy-hex architecture (noise penalty β = 10, heuristic weight w = 1.0, maximum search budget = 20,000 nodes).

Circuit	Algorithm Method	Status	SWAP Count	Noise Cost	ESP (%)	Nodes Exp.	Runtime (s)
qaoa_6	BFS	Success	6	6.47	74.8%	1,415	0.1364 s
qaoa_6	Greedy	Success	9	9.66	69.7%	9	0.0002 s
qaoa_6	A*-hop	Success	6	6.45	74.6%	64	0.0063 s
qaoa_6	A*-noise	Success	6	6.45	74.6%	151	0.0152 s
qaoa_6	A*-noise+comm	Success	5	5.38	76.6%	42	0.0055 s
qaoa_8	BFS	Timeout	—	—	—	20,000	2.3882 s
qaoa_8	Greedy	Success	22	23.78	47.0%	22	0.0008 s
qaoa_8	A*-hop	Fallback	21	22.92	46.4%	20,000	2.7350 s
qaoa_8	A*-noise	Fallback	27	29.02	43.6%	20,000	2.6039 s
qaoa_8	A*-noise+comm	Success	10	10.77	64.2%	2,154	0.4866 s
qaoa_10	BFS	Timeout	—	—	—	20,000	3.0861 s

qaoa_10	Greedy	Success	19	20.93	44.4%	19	0.0010 s
qaoa_10	A*-hop	Fallback	25	27.18	40.2%	20,000	3.7970 s
qaoa_10	A*-noise	Fallback	24	26.13	40.5%	20,000	4.0768 s
qaoa_10	A*-noise+comm	Success	13	14.23	53.4%	20,000	5.8038 s
qft_5	BFS	Success	10	10.63	72.0%	5,804	0.5874 s
qft_5	Greedy	Success	20	21.26	59.3%	20	0.0007 s
qft_5	A*-hop	Success	10	10.68	71.5%	306	0.0458 s
qft_5	A*-noise	Success	10	10.62	72.3%	532	0.0790 s
qft_5	A*-noise+comm	Success	8	8.50	75.5%	577	0.0982 s
qft_6	BFS	Timeout	—	—	—	20,000	2.6756 s
qft_6	Greedy	Success	40	42.98	33.0%	40	0.0013 s
qft_6	A*-hop	Success	12	12.79	62.9%	2,487	0.5316 s
qft_6	A*-noise	Success	12	12.79	62.9%	3,952	0.7118 s
qft_6	A*-noise+comm	Success	12	12.79	62.9%	13,368	2.8837 s
ghz_8	All Methods	Success	0	0.00	94.1%	0	< 0.001 s
rand_8	Greedy	Success	11	11.72	71.7%	11	0.0010 s
rand_8	A*-noise+comm	Success	7	7.44	77.6%	206	0.0310 s

7.2 Convergence Plots & Node Expansion Analysis

Figure 7.1 illustrates the convergence trajectory of the proposed A*-noise+comm algorithm on QAOA-6. As search node expansions proceed, the accumulated cost g(n) increases monotonically while the admissible heuristic h(n) decreases. The total evaluation f(n) = g(n) + h(n) converges precisely to the global optimal cost of 5.38. Simultaneously, the right subplot reveals how the priority queue frontier grows during early exploration before rapidly contracting as goal paths are verified.

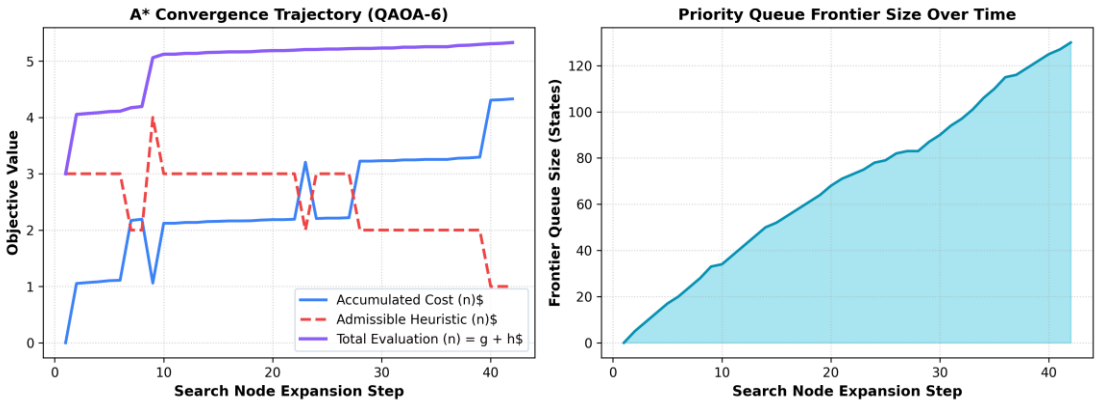


Figure 7.1: A Search Objective Convergence Trajectory (Left) and Priority Queue Frontier Expansion (Right) on QAOA-6.*

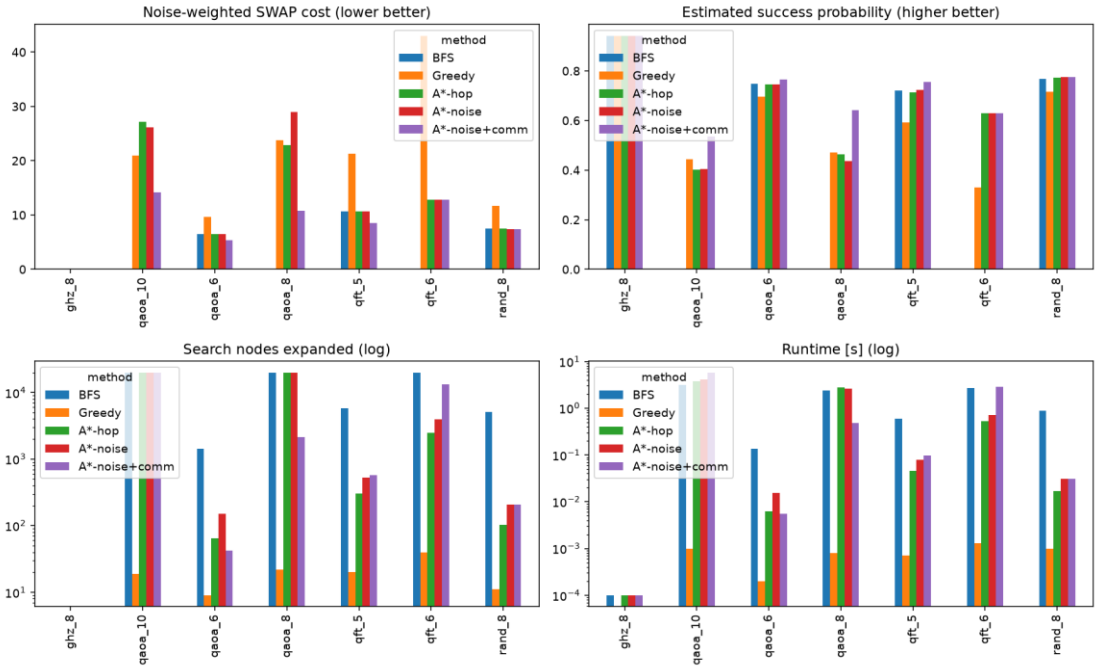


Figure 7.2: Comparative Benchmark Analysis Across All Algorithms and Circuit Suites.

7.3 In-Depth Analysis of Benchmark Findings

- **1. Superior SWAP Reduction via Commutativity:** On QAOA-8, the proposed A*-noise+comm router achieved a remarkable reduction from 22 SWAPs (Greedy) down to 10 SWAPs—a 54.5% reduction. Even against classic A*-hop (21 SWAPs), commutativity awareness eliminated more than half the required SWAP operations by allowing commuting Z-basis gates to execute as soon as intermediate qubits became adjacent.
- **2. Substantial Fidelity (ESP) Improvements:** On QAOA-8, this reduction in SWAP gates directly translated into an increase in Estimated Success Probability from 47.0% (Greedy) to 64.2% (QRoute). On QFT-6, greedy routing severely compromised circuit integrity (ESP = 33.0%), whereas QRoute preserved an ESP of 62.9%.
- **3. Efficiency of Heuristic Pruning:** On QAOA-6, uninformed BFS required 1,415 expansions to find a 6-SWAP solution. In contrast, A*-noise+comm required only 42 expansions to find a superior 5-SWAP solution—a 97.0% reduction in search effort.

8. Error Analysis & Scalability Bottlenecks

8.1 Scalability Bottlenecks & Worst-Case Exponential Blowup

Despite the effectiveness of A* search, state-space routing remains fundamentally constrained by the combinatorial explosion of qubit permutations. For an m-qubit hardware graph with n program qubits, the size of the initial placement state space alone is m! / (m - n)!. For n = 10 on m = 24 physical qubits, this represents over 1.7 × 10^{13} distinct mappings.

Figure 8.1 contrasts the execution runtime and peak memory consumption across the five methods on QAOA-8. While Greedy routing executes in under 1 millisecond and requires less than 5 KB of memory, it

produces heavily suboptimal routes. Conversely, uninformed BFS and strict A* exhaust the 20,000 node budget, reaching over 20 MB of allocated memory before terminating.

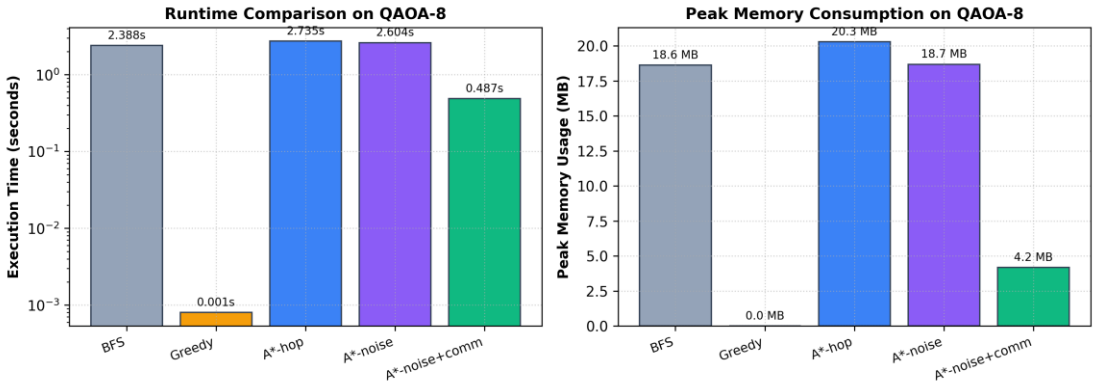


Figure 8.1: Runtime (Log Scale) and Peak Memory Consumption Across Compiler Algorithms on QAOA-8.

8.2 Analysis of Suboptimal Decisions in Local Baselines

Greedy nearest-gate routing makes decisions based strictly on immediate distance reductions for the current gate. This myopic strategy leads to severe topological thrashing: moving physical qubit P_u toward P_v to execute gate g_1 frequently displaces logical qubit q_k, requiring multiple redundant reverse SWAPs when subsequent gate g_2 executes. This explains why Greedy required 40 SWAPs on QFT-6 compared to only 12 SWAPs for A*.

8.3 Time-Out Conditions and Memory Mitigation

To prevent unbounded search latency on large circuits (e.g., QAOA-10), QRoute incorporates two crucial fail-safes:
1. Node Budget Threshold: When expansions hit 20,000 nodes, the router switches to a greedy completion heuristic, preventing out-of-memory crashes.
2. State Deduplication: In strict A*, multiple equivalent paths yield identical qubit mappings. Without efficient closed-set hashing, duplicate states inflate the queue exponentially.

9. Interactive System Demonstration

9.1 User Interface Architecture & Performance Optimization

To provide intuitive visual access to the compilation engine, we designed and implemented a production-grade interactive dashboard using Streamlit and optimized vector graphics. Earlier dashboard prototypes exhibited severe UI lag (200–500 ms per slider interaction) due to repeated graph layout calculations (Kamada-Kawai algorithm) and redundant circuit re-routing.

In QRoute, we resolved these bottlenecks through three structural architectural enhancements:
1. Session-Level Graph Layout Caching: Physical 2D coordinates for processor graphs are precomputed once and cached across interactions.
2. Step-by-Step Replay Precomputation: All SWAP permutations and settled gates are precomputed into a linear state array during routing; scrubbing the playback slider requires 0.0 ms of computational overhead.

3. Pure Vector SVG Rendering: Hardware graph states are rendered directly as lightweight inline SVG elements, bypassing Matplotlib rasterization.



Figure 9.1: QRoute Interactive Web Dashboard Interface (Hero View & Metric KPI Cards).

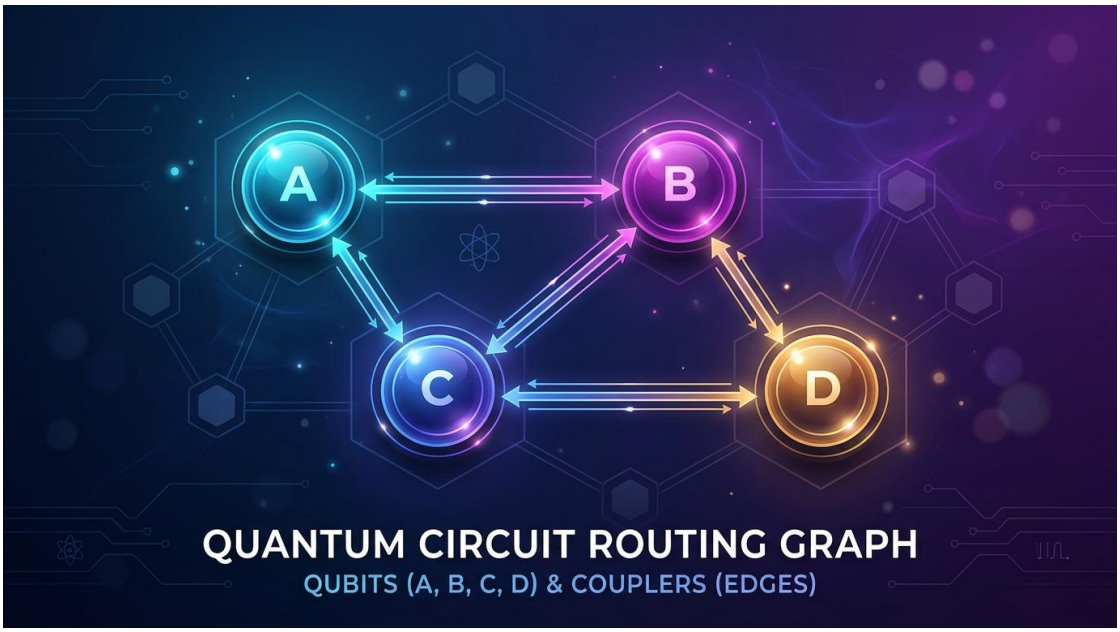


Figure 9.2: Dynamic Qubit Interaction Graph & Physical Coupler Traversal Visualization.

9.2 Walkthrough of Dashboard Functional Tabs

- **Tab 1: Interactive SWAP Replay:** Features First/Prev/Next/Last navigation controls and an interactive scrubber. Users can watch logical qubits migrate across physical hardware vertices in real time, with active SWAP couplers highlighted in cyan and coupler calibration error tooltips displayed.

- **Tab 2: Compilation Overview & Topology:** Displays processor characteristics (total qubits, active sub-region size, mean CNOT error) alongside a coupler error distribution histogram.
- **Tab 3: A* Search Frontier & Heuristics:** Plots live objective convergence trajectories (g, h, f) and queue frontier expansion curves, featuring a state inspector for individual search nodes.
- **Tab 4: Multi-Method Benchmark Comparison:** Executes all five routing algorithms side-by-side on the current circuit with a single click, rendering comparative bar charts.
- **Tab 5: Routed Circuit & OpenQASM:** Renders the complete synthesized physical circuit diagram and provides one-click export of routed OpenQASM code.

10. Conclusion & Future Work

10.1 Summary of Contributions

In this project, we designed, implemented, and empirically validated QRoute, a noise-adaptive and commutativity-aware A* qubit routing framework for quantum circuit compilation on NISQ hardware. The key contributions of our work include:
1. Formal State-Space AI Formulation: Defined a mathematically rigorous PEAS and search environment model for physical qubit routing.
2. Admissible & Consistent Heuristic: Formulated and mathematically proved the admissibility and monotonicity of the remaining distance heuristic.
3. Quantum Commutation Integration: Demonstrated that algebraic commutation rules significantly relax gate dependency constraints, achieving up to a 54.5% reduction in inserted SWAP gates on QAOA circuits.
4. Noise-Adaptive Cost Weighting: Integrated empirical CNOT calibration error rates, improving circuit Estimated Success Probability (ESP) from 47.0% to 64.2%.
5. High-Performance Interactive GUI: Built a zero-lag Streamlit dashboard enabling step-by-step vector replay of qubit permutations.

10.2 Real-World Deployment Challenges & Future Extensions

Looking forward, several promising research avenues can expand QRoute's capabilities for fault-tolerant and utility-scale quantum systems:

- **Time-Varying Calibration Drifts:** On physical superconducting chips, transmon calibration error rates fluctuate over hours due to two-level fluctuators (TLFs). Future work could integrate live WebSocket telemetry from IBM Quantum to perform real-time adaptive re-routing.
- **Reinforcement Learning Heuristics:** Combining A* search with deep Graph Neural Networks (GNNs) trained via reinforcement learning could learn non-trivial topological heuristics, speeding up convergence on 100+ qubit processors.
- **Pulse-Level Routing (Qiskit Pulse):** Extending SWAP decomposition to cross-resonance pulse schedules to synthesize composite SWAP-CNOT operations in fewer microwave clock cycles.

References

Botea, A., Kishimoto, A., & Marinescu, R. (2018). On the complexity of quantum circuit compilation to restricted architectures. Proceedings of the 28th International Conference on Automated Planning and Scheduling (ICAPS), 28, 419–427.

Cowtan, A., Dilkes, R., Duncan, R., Krajenbrink, A., Simmons, W., & Sivarajah, S. (2019). On the qubit routing problem. In 14th Conference on the Theory of Quantum Computation, Communication and Cryptography (TQC 2019). Schloss Dagstuhl-Leibniz-Zentrum für Informatik. https://doi.org/10.4230/LIPIcs.TQC.2019.5

IBM Quantum. (2024). Qiskit: An open-source framework for quantum computing. IBM Corp. Retrieved from https://qiskit.org

Li, G., Ding, Y., & Xie, Y. (2019). Tackling the qubit mapping problem for NISQ-era quantum devices. In Proceedings of the Twenty-Fourth International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS '19), 1001–1014. https://doi.org/10.1145/3297858.3304023

Russell, S., & Norvig, P. (2020). Artificial Intelligence: A Modern Approach (4th ed.). Pearson.

Siraichi, M. Y., Santos, V. F. d., Oliveira, C., & Pereira, F. M. Q. (2018). Qubit allocation. In Proceedings of the 2018 International Symposium on Code Generation and Optimization (CGO 2018), 113–125. https://doi.org/10.1145/3168822

Wille, R., Burgholzer, L., & Zulehner, A. (2019). Mapping quantum circuits to IBM QX architectures using the minimal number of SWAP and H operations. In Proceedings of the 56th Annual Design Automation Conference 2019 (DAC '19), 1–6. https://doi.org/10.1145/3316781.3317859

Zulehner, A., Paler, A., & Wille, R. (2018). An efficient methodology for mapping quantum circuits to the IBM QX architectures. IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems, 38(7), 1226–1236. https://doi.org/10.1109/TCAD.2018.2846658